Reviewer 1

The authors describe a new bioconductor Rpackage: MSP-MS that is useful for analyzing peptides resulting from protease cleavage experiments. Overall it made a good impression to me and should be useful to the respective scientific community and I recommend moving this forward for publication.

We thank Reviewer 1 for their positive evaluation and for sharing their experience using the software. We appreciate their constructive suggestions and are happy to incorporate them to further our goal of making *mspms* analysis as accessible and reproducible as possible, regardless of users' computational environments.

Content remarks:
The definition of the percentage of undetected cleavage_product is unclear to me and how it was determined. Eg. Is this the number of uncleaved peptides / all detected peptides?
Maybe the same or an additional feature - is there a way to see which library peptides were not cleaved? I only see "cleavage_product" and "full_length" as options to select from.

We thank the reviewer for this helpful clarification request. The *percentage of undetected cleavage product* represents the proportion of samples in which a given peptide cleavage product was not detected. A *cleavage product* refers to a peptide generated through proteolytic cleavage of a peptide from the MSP-MS library. Importantly, certain full-length peptides (14 amino acids long) may not be detected due to their physicochemical properties and poor ionization efficiency, even though shorter products resulting from their cleavage may be readily observed.

To clarify this, we have added the following **Quality Control Metrics** section to the *Methods*:

Quality Control Metrics
mspms computes two quality control (QC) metrics to assist users in evaluating MSP-MS data quality.
(1) The first metric reports the percentage of samples in which a given peptide from the library—either a full-length peptide or a cleavage product—is not detected. Full-length peptides are included because certain sequences may exhibit poor ionization and thus remain undetected, even if their corresponding cleavage products are observed.
(2) The second metric reports the percentage of the entire library that is undetected within each sample.
Together, these QC metrics help users diagnose issues related to data quality, ionization efficiency, or peptide preparation in their MSP-MS experiments.

Software remarks:

1. I had installation problems using biocManager::install and only managed via devtools::install_from_github - please double check

We thank the reviewer for noting this critical installation issue. To investigate, we tested installations on multiple platforms using fresh R environments, and our findings are summarized below.

On a macOS system, installation via BiocManager::install() was successful after updating to the most recent Bioconductor release (3.22).

On a Windows system (R version 4.5.2), however, installation initially failed with the following error:

S4Vectors 0.47.0 is being loaded, but > 0.47.6 is required.

Upon further inspection, the most recent version available is S4Vectors 0.47.8, but precompiled Windows binaries for this version have not yet been distributed through Bioconductor. Consequently, Windows systems automatically fall back to the older 0.47.0 release, leading to dependency conflicts.

To confirm this diagnosis, we manually installed S4Vectors from source using:

BiocManager::install("S4Vectors", ask = FALSE, type = "source")

This successfully installed version 0.47.8, after which installation of *mspms* completed without issue.

In summary, the installation failure appears to stem from a short-term delay in Bioconductor's distribution of updated **S4Vectors** binaries for Windows. The latest Bioconductor release was issued on **October 30, 2025**, and we expect that updated binaries will become available shortly, resolving this issue for new users.

2. Once installed via devtools the library "ImputeLCMD" was still missing and had a R version conflict (requires R<4.4.1) however your tool mspms required R>=4.4.0 … I didn't test all combinations and went on but this might be an issue you want to ease out or solve

During the Bioconductor review, it was suggested that imputeLCMD removed from **Imports** (https://github.com/Bioconductor/Contributions/issues/3460).

Upon further consideration, we now realize this recommendation was not appropriate, as imputeLCMD is **required for our core imputation strategy**. As such, we have moved imputeLCMD back to **Depends** to ensure the package functions correctly.

3. iceLogo & Cleavage Count not working in shiny app

We are unsure what the reviewer is referring to here. On the version of **mspms** hosted on **shinyapps.io**, both iceLogo and Cleavage Count work as expected in our testing, and should for everyone since it is the same deployment.

To determine whether the reviewer was referring to a locally installed version of **mspms-shiny**, we tested a fresh install of the application on a new computer. In our testing, all features functioned correctly. We suspect that this may have been an issue with the previously mentioned imputeLCMD issue, but it is hard for us to tell without more information.

4. If one just wants to access the peptide SummarizedExperiment lists from the processed_qf - is there a way to look at them easily in R? Maybe you want to describe this as well in the vignette

Yes, this can be easily accomplished by subsetting the QFeatures object.

For accessibility to new users, information has been added to the vignette in the *Accessing data stored in QFeatures* section.

Minor text remarks:
1. The section of the iceLogo Analysis contains a lot of brackets and I don't understand the logic. Maybe typos? Or just the way it is visualized inthe review system?

After inspection, we believe that what the reviewer is referring to is due to the way the math was visualized in the review system. As such, we have made no changes to specifically address this, although the iceLogo methods have been revamped in response to reviewer 3s comment.

2. The Wichura algorithm could be cited

This was an oversite on our part. The Wichura algorithm has now been cited.

3. The QRILC algorithm is implemented in another package ImputeLCMD, which the library you employ MScoreutils is calling in the backend. You might want to cite this package as well

The relevant citation has been added.

4. The heatmap x-axis should be labeled “peptides” in Fig2B

This has been addressed. In addition, the y axis has been named samples, and the color scale has been updated to reflect that these values represent scaled intensities.

Reviewer 2

In this manuscript, Charlie Bayne et al introduced mspms, an R package for Multiplex Substrate Profiling by Mass Spectrometry (MSP-MS) data analysis. This tool streamlines the MSP-MS data analysis workflow by providing modules of data preprocessing, normalization, statistical testing, visualization, and quality control. All the workflow parts have been integrated within the Bioconductor framework, and could also be accessed via graphical user interface in Shiny APP for users without programming experience. The authors also validated their tool by using published raw data from four well-characterized cathepsins (A–D). With the quantification results from PEAKS, Proteome Discoverer, and FragPipe, they confirmed the tool’s reliability with reproduction of known enzymatic preferences. The code and usage guidance are clear at the GitHub/Shiny App pages. Although the algorithms are not novel in this tool, but the building and application of a standardized workflow for MSP-MS data analysis is valuable and the open-sourced tool provides a helpful resource for further development by researchers in the protease research field. Therefore, it is recommended that the manuscript be published after minor revision

We thank reviewer 2 for finding our tool valuable for researchers in the protease research field. We look forward to incorporating their feedback in order to improve our software and manuscript.

Minor:
1. The supplementary figure indexing and citation in this manuscript are out of order. The reindexing and revision of supplementary figure are required.

We thank the reviewer for catching this. The indexing has been updated.

2. The first paragraph in Discussion section: “Furthermore, only data exported from the proteomic search engine PEAKS was compatible, hindering usability across different research groups.” The sentence is confusing. The authors should point out what was PEAKS compatible with.

We thank the reviewer for pointing out this confusing sentence. We have revised the text from

“Furthermore, only data exported from the proteomic search engine PEAKS was compatible, hindering usability across different research groups.”

to

“Furthermore, these scripts were specifically designed to process data exported from the proteomic search engine PEAKS, preventing their use across research groups that employed different platforms.”

3. The algorithm description in iceLogo analysis: “We implemented this approach in R, using the previously described Java implementation as a reference.” There is no citation or description for the reference. Please provide essential information of the reference.

This was an oversight. We have now added the appropriate reference to that section of the text.

4. The discussion part is too long and focuses too much on the descriptions about the advantages, and this should be shortened. Although many advancements of the tool have been presented in this manuscript, the discussion about the disadvantages or limitations of their tool are also recommended.

We appreciate the reviewer’s suggestion. In the revised version, we have added a paragraph outlining current limitations and future directions.

Despite its advantages, mspms has several important limitations. First, it currently operates on result files from specific proteomic search engines. While we have incorporated parsers for several commonly used platforms, compatibility is not universal. However, because the input structure is standardized, new parser functions can be readily implemented to support additional search engines in future releases, leveraging the core functionality of the mspms platform.

Second, mspms currently supports the Multiplex Substrate Profiling by Mass Spectrometry (MSP-MS) workflow but does not yet incorporate the quantitative TMT-based extension (qMSP-MS), which has been shown to minimize experimental and instrument-derived variance while improving assay throughput (31).
Finally, mspms is currently limited to DDA-based acquisition strategies. Extending compatibility to data-independent acquisition (DIA) formats represents an important future direction. We anticipate that applying DIA-based workflows to MSP-MS experiments, particularly in combination with recent technological advances in mass spectrometry hardware, could facilitate shorter chromatographic gradients and higher-throughput analyses while reducing instrument time.

Reviewer3

The manuscript introduces *MSPMs*, an R/Bioconductor package with a Shiny interface for standardized analysis of Multiplex Substrate Profiling by Mass Spectrometry (MSP-MS) data.

The authors claim it is the first dedicated, publicly available software for MSP-MS, addressing the current reliance on ad hoc, non-reproducible workflows. The tool integrates established Bioconductor data structures, implements normalization, imputation, statistical testing, and visualization functions, and includes an R-based reimplementation of the *iceLogo* algorithm for motif analysis. Validation using cathepsin datasets demonstrates that *mspms* reproduces known substrate specificities. Overall, the work's novelty lies in providing a reproducible, accessible, and extensible platform for MSP-MS data analysis.

We thank the reviewer for their extremely thoughtful and in-depth review and look forward to incorporating their expert advice to improve mspms.

Concerns:

Introduction

The claim that mspms is "the first publicly available platform for MSP-MS data analysis" may be overstated. Before mspms, MSP-MS data analysis relied on several publicly available tools and pipelines. For example, the MSP-Xtractor software (publicly downloadable and cited in numerous papers) for cleavage extraction, doi: 10.1016/j.cgh.2015.08.038, and the UCSD lab's R scripts (shared via GitHub) for statistical analysis https://github.com/briannahurysz/MSP-MS. Moreover, a standardized analysis protocol was published in 2023, effectively making the know-how publicly accessible: https://doi.org/10.1038/s41598-025-11635-1.

We thank the reviewer for highlighting existing tools such as *MSP-Xtractor* and the lab's own legacy R scripts. We agree that these resources have provided valuable functionality for MSP-MS data analysis in the past. However, *mspms* differs in being a unified, fully documented, and easily installable Bioconductor package that consolidates multiple analysis steps—including preprocessing, normalization, visualization, and statistical testing—into a single, coherent framework. The inclusion of an interactive Shiny graphical interface further distinguishes *mspms*, expanding accessibility for the broader protease research community. Based on our first-hand experience, none of the previously available solutions constitute a comprehensive platform for MSP-MS data analysis.

To address the reviewer's concern, we have revised the text to state:

"*mspms* is the first publicly available **comprehensive** platform for MSP-MS data analysis downstream of peptide identification and quantification."

We believe this revised wording more accurately reflects the novelty of *mspms* by emphasizing its integrated functionality, ease of use, and broad accessibility compared with earlier standalone tools and partial pipelines.

We have also added a brief mention of MSP-Xtractor to the introduction of the paper.

While the authors convincingly motivate the need for a open source MSP-MS analysis tool, they should summarize how MSP-MS data were analyzed in previous publications (e.g., O'Donoghue et al. 2012; Jiang et al. 2021) as well as with: 10.1016/j.cgh.2015.08.038, https://github.com/briannahurysz/MSP-MS or https://doi.org/10.1038/s41598-025-11635-1 , even if those relied on custom or unpublished scripts. This information would contextualize their claim that no dedicated software previously existed and highlight the advances represented by MSPMs.

We thank the reviewer for pointing out these important points. We have attempted to explicitly expand on the way that MSP-MS data was previously handled and shortcomings of that approach that mspms was specifically developed to address in the introductory paragraph.

"MSP-MS data analysis has historically relied on MSP-Xtractor software (https://pharm.ucsf.edu/craik/research/extractor) and/or custom, sparsely documented scripts. MSP-Xtractor provides basic functionality for extracting cleavage sites and counting spectral events from peptide identifications generated by ProteinProspector(5), but it lacks integration with downstream normalization, statistical testing, or visualization tools. Custom scripts extended the analysis to include imputation and basic statistical testing but depended heavily on manual steps, including exporting data to spreadsheets, editing hardcoded variables, and uploading processed results to the IceLogo graphical interface for visualization"

In addition, the authors should clarify **why MSP-MS data are analyzed using label-free intensity-based quantification** rather than alternative approaches such as spectral counting. Since the peptide library is well-defined and lacks protein-level inference, the rationale for using continuous intensity measures rather than discrete spectral events is not immediately evident. A brief justification—such as improved dynamic range, better suitability for short synthetic peptides, or compatibility with downstream normalization and imputation—would help readers understand the analytical design and strengthen the study's methodological transparency.

We thank the reviewer for this comment. **mspms** uses label-free intensity-based quantification. While previous MSP-MS studies have employed both spectral counting and intensity measurements, modern high-resolution mass spectrometers provide a wide dynamic range and precise peptide quantification. Intensity-based values therefore offer a more accurate and quantitative measure of peptide abundance than spectral counts, which is why this approach is implemented in **mspms**.

Furthermore, the manuscript would benefit from a concise explanation of the choice of upstream proteomics software. The authors employ three search platforms, PEAKS, Proteome Discoverer, and FragPipe, but the rationale for this selection is not discussed. While FragPipe is freely available for academic use, it remains closed source, as do all three tools. Clarifying whether the goal was to benchmark *mspms* against commonly used, non-open-source solutions or to demonstrate compatibility with existing workflows would help readers understand the scope of the validation. Given the manuscript's emphasis on transparency and reproducibility, it would also be valuable to include at least one fully open-source pipeline (e.g., Comet coupled with FlashLFQ or Sage) in the comparison to illustrate how *mspms* performs within a genuinely open and reproducible analysis framework.

Our choice to implement support for PEAKS, Proteome Discoverer, and FragPipe was guided by their widespread adoption in the proteomics community as well as our prior experience with these platforms. We fully agree with the reviewer that inclusion of a fully open-source workflow aligns with the manuscript's emphasis on transparency and reproducibility. To address this, we have added an mspms parser compatible with Sage, enabling evaluation within a completely open and reproducible analysis framework.

Interestingly, we observed dramatic differences in the correlation of detected peptide intensities for Sage compared to the other software solutions ($R^2 \sim 0.2$ for Sage vs. $R^2 > 0.8$ for others; S1). However, despite these differences, substrate specificities for Cathepsins A–C were largely consistent with those observed using the other software tools (updated S1). We note that Sage had the worst ability to detect Cathepsin D's known endopeptidase activity (updated S1), which likely reflects a reduced ability to detect very short peptides with the search settings applied (updated S3).

Material and Methods

In the "Data used for Study" section, the description of the peptide library is insufficient. Given that the library defines the entire MSP-MS assay, the authors should summarize its key characteristics (e.g., that it consists of 228 rationally designed 14-mer peptides, covering diverse amino acid contexts)

We have expanded on our original text in order to emphasize the key characteristics. "In brief, these data were generated from a study that utilized a 228-member, rationally designed 14-mer peptide library covering diverse amino acid contexts."

In the **Preprocessing** section, the authors refer to "the scissile bond" without defining the term. As this concept is central to protease specificity analysis, a short explanatory sentence—e.g., "the scissile bond refers to the peptide bond cleaved by the protease (between the P1 and P1′ residues)", would improve clarity, especially for readers from the computational proteomics community.

We have incorporated this suggestion to improve clarity. "Cleavage motifs of a user-specified length on the N and C terminal sides of the scissile bond (the peptide bond specifically cleaved

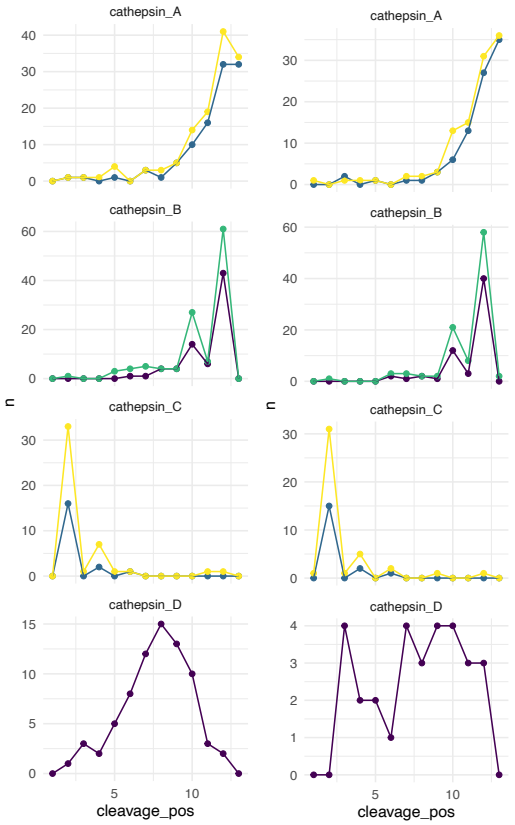
by the protease, located between the P1 and P1′ residues) are calculated, and the numerical position of each cleavage site is determined with reference to the peptide from which it originated.”

In the **Data Processing** section, the described workflow includes $\log_2$ transformation, normalization, imputation, and then "reverse $\log_2$" transformation. It is unclear why the data are exponentiated again, as statistical analysis (including t-tests and limma) should usually be performed on the $\log_2$-transformed scale to stabilize variance. The authors should clarify whether this back-transformation is applied only for visualization or report generation, or for downstream testing, since the latter would not be appropriate.

We thank the reviewer for raising this important point.

Our original workflow followed historical MSP-MS data processing conventions, which utilized reverse $\log_2$–transformed values for visualization and reporting. We agree that, conceptually, statistical tests such as t-tests should be applied to $\log_2$-transformed values to stabilize variance.

To verify our approach, we reviewed our code and discovered that t-tests were indeed being performed on the non–$\log_2$-transformed values. Examination of the legacy R scripts (Rohweder et. al) revealed that they also used reverse $\log_2$–transformed values for statistical testing. We therefore updated the code to perform t-tests on $\log_2$-transformed values and evaluated the impact on the results. For comparison, results from t-tests on the reverse $\log_2$–transformed values are shown on the left, and results from t-tests on $\log_2$-transformed values are shown on the right.



This analysis showed that the reverse log_2 transformation produced near-identical results for all cathepsins except cathepsin D, the only endopeptidase evaluated in our study. We suspect these deviations are related to the shorter peptide products generated by endopeptidase cleavage. Consequently, we have updated our code to perform t-tests on log2 transformed values.

Furthermore, we have updated the manuscript and figures to present results from limma-based statistical testing on log_2-transformed values. This approach is better suited for MSP-MS datasets and provides substantial speed improvements over the legacy t-test workflow, which was included in the original manuscript because the limma method had not yet been implemented at the time of writing.

In the **Data Processing** section, the authors indicate that a new SummarizedExperiment is created for each preprocessing step ("log2_peptides", "log2_peptides_norm", etc.), which appears inconsistent with recommended QFeatures usage, where multiple processing stages are typically stored as separate assays within a single SummarizedExperiment object. The authors should clarify this design choice.

We respectfully disagree with the reviewer on this point. Our approach is consistent with the recommended usage of QFeatures, in which multiple preprocessing stages are represented as separate SummarizedExperiment objects stored within a single QFeatures container. In fact, a SummarizedExperiment object can hold only one assay, which was part of the motivation for developing higher-level container classes such as QFeatures and MultiAssayExperiment.

iceLogo

The *iceLogo* analysis is a central methodological and interpretative component of this manuscript. It is one of the key innovations of the *mspms* package, with an R-based reimplementation of the original Java algorithm for visualizing amino acid preferences around the protease cleavage site. The approach is repeatedly referenced throughout the paper—in the Methods section as a distinct analytical module, in the Results, where multiple figures (e.g., Figure 3D and Supplementary Figures 2 and 6) rely on it for interpreting substrate specificity, and again in the Discussion, where it is highlighted as a significant feature of the software. Because *iceLogo* plots form an essential part of the package's novelty and the interpretation of biological results, the corresponding Methods description should be rewritten for clarity and completeness.

The *iceLogo Analysis* section is unclear in its current form because:

- The description uses **ambiguous phrasing** (e.g., "a count of the number of times an amino acid at each position is calculated"), making it difficult to follow the computational steps.
- **Key terms are undefined**, such as what constitutes the "experimental" and "reference" sets or what the "entire set" refers to when computing frequencies.

- The **statistical procedure** (standard deviation calculations, p-values, and thresholds) is described narratively, without clear equations or variable definitions.
- Lack of reference to the Wichure Algorithm. Also, a sentence explaining how the algorithm obtains p-values would be helpful
- It is **unclear how the background frequencies** are derived (from all possible scissile bonds, the peptide library, or observed peptides).
- The **link between computed values and visual representation** (how fold or percent change translates into letter heights in the logo) is not clearly explained. Including an example figure might help.
- Overall, the section lacks a clear, stepwise explanation that would allow a reader to reproduce or verify the R-based reimplementation.

We thank the reviewer for their detailed feedback regarding the *iceLogo* analysis description. We agree that this section is central to the manuscript and have revised it extensively to improve clarity, precision, and reproducibility. The revised version now defines all key terms (experimental and reference sets), outlines the computational procedure step by step, explains how background frequencies are derived, clarifies how statistical significance is calculated, and explicitly describes how results are translated into the logo visualization. The new version appears below.

iceLogo Analysis

To visualize substrate amino acid preferences surrounding protease cleavage sites, we reimplemented the **iceLogo** algorithm in R(22). This approach statistically compares amino acid frequencies at defined positions relative to the scissile bond between an experimental set and a reference set and visualizes the differences as letter heights in a logo.

1. Definition of sets

Experimental set: User-defined, representing the cleavage sequences of interest. In our analyses, this includes significantly altered peptides (p.adj < 0.05 and $\log_2$ fold change > 3) relative to time 0.

Reference set: Comprises all possible cleavage sequences present in the MSP-MS peptide library.

2. Frequency calculation

For each position within a user-defined amino acid window around the scissile bond, the count and frequency of each amino acid are computed separately for both the experimental and reference sets.

3. Statistical testing

For each amino acid at each position, the standard deviation (σ) is calculated from the reference set frequency (f%):

$$\sigma = \sqrt{\frac{f\%}{N}}$$

where N represents the sample size, i.e., the number of peptides in the **experimental set** at that position.

The Z-score is then calculated as:

$$Z = \frac{X - \mu}{\sigma}$$

where X is the frequency of the amino acid at that position in the experimental set, and μ is the corresponding frequency in the reference set. This Z-score indicates how many standard deviations the observed frequency of the experimental set deviates from that of the reference set.

Significance is subsequently determined by whether the Z-score falls outside the confidence interval, with the Wichura algorithm (23) used to convert Z-scores into p-values. Only amino acids with p-values less than or equal to the user-specified threshold are retained for visualization..

4. Visualization

Significant differences between the experimental and reference frequencies are visualized as letter heights using the **ggseqlogo** R package . Users may choose either **percent change (PC)** or **fold change (FC)** to define the height of each amino acid letter:

$$PC = F^{+} - F^{-}$$
$$FC = \frac{F^{+}}{F^{-}}$$

Where F^+ and F^- are the frequencies in the experimental and reference sets, respectively.

Fold changes smaller than 1 are converted to maintain positive/negative directionality:

$$FCcon = \frac{1}{FC} \times -1$$

5. Handling Extreme Values

If only one amino acid is significant at a position and its calculated letter height is infinite, the height is set to the maximum allowable value in the logo.

If multiple amino acids at a given position are all significant and have infinite calculated heights, their combined height is capped at the maximal value displayable in the iceLogo plot.

Discussion

Figure 3A

Missing axis labels: The y-axes in the center and left columns (volcano plots and cleavage-position plots) lack labels, making it unclear what quantity is plotted. For instance, the cleavage-position plots should specify whether the y-axis represents "number of significant cleavage events" or "frequency."

To avoid over cluttering the figure, we opted not to label each individual y-axis; instead, a shared scale is provided in the center of each panel, indicating $-\log_{10}$(p.adj) values for the volcano plots and counts (n) for the cleavage-position plots. We have updated the legend in Panel C to explicitly indicate “number of significant cleavage events” rather than simply “n” to clarify the quantity being plotted.

Figure 3A, described in the legend as *"Summarized substrate specificities for cathepsin A–D as reported in the literature"*, is never referenced in the text. There is no sentence in the Results or Discussion that directs the reader to panel A or explains its purpose, i.e., that it provides the literature-based reference for comparison with panels B–D.

We respectfully disagree with the reviewer’s assessment. The results section explicitly references Figure 3A in the sentence: “To evaluate protease activity relative to reported substrate specificities (Figure 3A), …” This sentence directs the reader to the panel and clarifies its role as the literature-based reference for comparison with panels B–D.

In Figure 3D, the *iceLogo* panels are challenging to interpret. It is not clear why residues denoted as "X", which usually represent missing or terminal positions, appear on the *left* (N-terminal side) of the logo, even though cleavage, for cathepsin A and B, is described as occurring on the *right* (C-terminal side).

We thank the reviewer for raising this point regarding the presence of “X” residues in the iceLogo panels. This character represents positions beyond the N- or C-terminal ends of the peptide. While these positions are not typically included in traditional iceLogo visualizations, we have chosen to retain them because they provide informative context about cleavage near peptide termini, allowing readers to interpret exopeptidase specificity more completely. This approach

ensures that both endopeptidase and exopeptidase activities are visually represented in a consistent framework.

For example, it allows the user to readily identify C-terminal exopeptidase activity (Cathepsin A, Fig. 3D), C-terminal dipeptidyl carboxypeptidase activity (Cathepsin B, Fig. 3D), N-terminal dipeptidyl aminopeptidase activity (Cathepsin C, Fig. 3D), and endopeptidase activity (Cathepsin D, Fig. 3D) all within a single iceLogo plot.

Moreover, the Methods do not state whether the *iceLogo* plots were generated using all detected cleavage sites or whether they were filtered for the most frequent or significant cleavage positions.

We have updated the Ice Logo Analysis methods to make our approach more transparent.

“For the iceLogo plots presented, we used only significantly altered peptides relative to time 0 as the experimental set. This approach focuses the analysis on cleavage events that show meaningful changes, rather than including all detected cleavage sites, to highlight biologically relevant motifs.”

It is unclear how the quantities "percent change (PC)" and "fold change (FC)" described in the Methods are used in the iceLogo plots. The authors should specify which metric underlies the logos shown in Figure 3D and Supplementary Figures, and clarify how statistical significance thresholds were applied in generating the letter heights.

We have clarified this with the following text in the appropriate figure legends.

“For the iceLogo plots reported, significantly altered peptides ($p < 0.05$, $\log_2FC > 3$) relative to time 0 were used as the experimental observations, and letter heights represent the percent change (PC) relative to the initial peptide library. Only residues with $p \leq 0.05$ are shown.”

Further comments

- Reduce repetition: Several sentences reiterate that *mspms* "reliably capture expected substrate specificities.", which could be stated once, followed by more concrete quantitative results.

We thank the reviewer for the suggestion. Upon review, the phrase highlighting that mspms reliably captures expected substrate specificities appears only once in the Abstract. We believe this is appropriate for emphasizing the tool’s validation without undue repetition.

- Add numerical context: Phrases like "numerous significantly upregulated peptides" could be replaced with counts or percentages to clarify.

We thank the reviewer for their suggestion. We have updated text as follows.

The results, visualized as volcano plots, revealed between 19 and 126 significantly upregulated peptides ($\log_2$ fold change ≥ 3, p.adj ≤ 0.05) after the incubation times assessed.

We have also edited the text in various other places in response to the reviewer's concern.

Ensure consistency in terminology: Terms such as *"cleavage sites," "positions," "significant peptides,"* and *"substrate specificity"* are sometimes used interchangeably; define them once and use them consistently.

We thank the reviewer for highlighting the need for consistent terminology. We have edited the text to be more consistent with our terminology.

- Highlight comparison outcomes: The comparison across PEAKS, PD, and FragPipe could be summarized more explicitly—e.g., with a concise table or numeric summary rather than only a qualitative description.

We thank the reviewer for this helpful suggestion. Much of the quantitative comparison between PEAKS, PD, and FragPipe is presented in Figure S1, but we have revised the main text to more explicitly summarize these outcomes, highlighting key numeric results to complement the qualitative discussion.

- Clarify supplementary figure references: Supplementary Figures 1–5 are cited but not always explained; the text should briefly summarize what each demonstrates (correlation, agreement, peptide length distribution, etc.).

We thank the reviewer for pointing this out. We have revised the manuscript to concisely summarize the contents of Supplementary Figures 1–5 within the main text, briefly describing what each figure demonstrates (e.g., correlation, agreement, and peptide length distribution).

Package Code

The *mspms* package exhibits strong attention to **testing**, with comprehensive test coverage and clear documentation. All exported functions are tested, including systematic validation of preprocessing and edge cases, ensuring reliability and maintainability. Documentation is well

structured, with runnable examples and a detailed vignette that demonstrates complete workflows across supported data formats.
The functional organization into four logical categories—preprocessing (prepare_* functions), processing (process_qf), statistics (limma_stats, log2fc_t_test), and visualization (plot_* functions)—creates an intuitive API that mirrors the natural analysis workflow.

We thank the reviewer for looking into the testing and are happy they found it sufficient and well organized.

Concerns

DRY Principle Violations

The most egregious **DRY** (Don't Repeat Yourself) violation occurs in the near-complete duplication of file validation logic across four functions — check_file_is_valid_peaks, check_file_is_valid_fragpipe, check_file_is_valid_pd, and check_file_is_valid_diann — located in lines 11–115 of *preprocessing_helper_functions.R*. Each follows the same pattern: defining expected columns, checking for missing columns using %!in%, and throwing formatted errors. Together, these blocks account for roughly **100 duplicated lines** of code.

We agree with the reviewer that our implementation was suboptimal and unnecessarily repeating. As such, I have rewritten the validation functions to use a shared check_file_is_valid() function which takes a data frame, the expected names of the columns, and the tool name as arguments.

Similarly, the four prepare_* functions repeat an identical five-step workflow (read file → validate → transform → load colData → convert to QFeatures), resulting in roughly **150 lines of redundant code**.

We thank the reviewer for bringing up this point. We also agree that our previous implementation of the four prepare_* functions resulted in redundant code. We have refactored the code to utilize a generic prepare_file() function that centralizes the core logic: validate → transform → load colData → convert to QFeatures.

Tool-specific transform_* functions have been introduced to handle the unique formatting requirements of each data type. The prepare_* functions themselves are retained to preserve the existing package API, ensuring backward compatibility while reducing code duplication and improving maintainability.

The nterm_cleavage and cterm_cleavage functions are about **95% identical**, differing only in direction (~80 duplicated lines). Finally, the *iceLogo* pivot operations in prepare_sig_p_dif and prepare_fc replicate **40 lines** of identical transformation logic.

We thank the reviewer for highlighting the structural similarity between the nterm_cleavage() and cterm_cleavage() functions, we agree that these are unnecessarily redundant.

This has been updated to rely on a generic cleavage function that centralizes the logic.

nterm_cleavage and cterm_cleavage are now lightweight wrapper functions in order to maintain the packages previous API.

Moreover, the current implementation contains two bugs. First, the error messages in the Proteome Discoverer and DIA-NN validators incorrectly reference "PEAKS file" rather than the appropriate software name (lines 86 and 111). Second, all four functions check if (length(missing_names) > 1) when they should check if (length(missing_names) > 0), meaning that files with exactly one missing column would incorrectly pass validation.

These issues in the validation logic have been addressed by implementing the reviewer's previous suggestion to centralize the validation logic.

Structural and Architectural Issues

The package exhibits **poor separation of concerns**. For example, the prepared_to_qf function (lines 344–404) mixes data transformation, validation, and QFeatures object creation within a single routine. Similarly, plotting functions such as plot_heatmap (lines 31–63) perform extensive internal data preparation rather than accepting preprocessed input, preventing users from inspecting or modifying the data before visualization.

We thank the reviewer for highlighting the separation of concerns. We agree that, from a software engineering perspective, separating data transformation, validation, and QFeatures object creation would improve modularity. However, prepared_to_qf and other convenience functions (e.g., plot_heatmap) were intentionally designed to streamline common workflows and produce ready-to-use outputs for typical users. This design prioritizes **usability and reproducibility**, while advanced users can still bypass these convenience functions and perform individual preprocessing or visualization steps manually. For these reasons, we have chosen to retain the current structure.

Error handling and messaging

Error handling and messaging vary widely in quality. Some messages are clear and informative (e.g., prepare_peaks.R, line 34, reporting filtered peptides with percentages), while others contain spelling or copy-paste errors referencing the wrong software type. Additionally, the file find_paired_peptides.R appears to contain **exploratory code**, including library(ggplot2) on line 28 — a command that should never appear in production package code — suggesting that this file belongs under inst/scripts/ rather than within the distributed codebase.

We thank the reviewer for identifying the issue with find_paired_peptides.R. This function was inadvertently included in the repository. It has now been removed from the distributed codebase.

In addition, we have systematically reviewed and improved all user- and developer-facing messages throughout the package striving to ensure they are consistent, informative, and free from typographical or copy-paste errors.

Performance and Implementation Quality

The function count_cleavages_per_pos (lines 148–168 in *plotting_helper_functions.R*) demonstrates a critical performance anti-pattern. It initializes an empty tibble using final <- tibble::tibble() and then iteratively appends rows inside a loop via final <- dplyr::bind_rows(final, out). This results in quadratic $O(n^2)$ complexity, as R must reallocate and copy the entire data frame at every iteration. The issue could be trivially avoided by pre-allocating a list and performing a single bind_rows() call after the loop completes.

We thank the reviewer for highlighting this performance issue. While the original implementation had minimal practical impact given the typical number of iterations in our datasets, we have updated the function following the reviewer's recommendation, pre-allocating a list and performing a single bind_rows() call after the loop. This improves efficiency and scalability.